3. Primary 노드 구현

3.1 Primary 노드 역할

Primary 노드는 Narwhal 시스템에서 컨센서스 로직을 담당하는 핵심 구성 요소입니다.

3.1.1 주요 책임

- **인증서 생성**: Worker로부터 받은 배치를 기반으로 헤더 생성
- **서명 수집**: 다른 Primary들로부터 헤더 서명 수집
- **DAG 관리**: 인증서들 간의 의존성 그래프 유지
- **컨센서스 실행**: Tusk 또는 Bullshark 프로토콜 실행

3.1.2 아키텍처 다이어그램

Primary 노드는 다음과 같은 구성 요소들로 이루어져 있습니다:

- Header Processing Pipeline
- Certificate Processing Pipeline
- Consensus Engine
- Network Interface

3.2 헤더 처리 파이프라인

3.2.1 헤더 생성 과정

```rust
// 헤더 생성의 핵심 로직
pub struct HeaderBuilder {
committee: Committee,
worker_cache: WorkerCache,
gc_depth: Round,
}

impl HeaderBuilder {
pub fn make_header(&self,
round: Round,
parents: BTreeSet<CertificateDigest>,
payload: IndexMap<BatchDigest, WorkerId>
) -> Header {
Header {
author: self.name.clone(),
round,
epoch: self.committee.epoch(),
parents,
payload,
}
}
}
```

3.2.2 부모 선택 알고리즘

- **Stake-weighted Selection**: 스테이크 가중치 기반 부모 선택
- **Liveness Guarantee**: 최소 2f+1개의 부모 참조 보장
- **Causal Ordering**: 인과관계 유지를 위한 의존성 체크

3.3 인증서 처리 파이프라인

3.3.1 서명 수집 메커니즘

```
pub  struct SignatureCollector {
committee: Committee,
pending_headers: HashMap<HeaderDigest, PendingHeader>,
}

struct PendingHeader {
header: Header,
signatures: Vec<(PublicKey, Signature)>,
created_at: Instant,
}
```

3.3.2 인증서 완성 조건

- **임계값 달성**: f+1개 이상의 유효한 서명 수집
- **서명 검증**: 각 서명의 암호학적 유효성 확인

- **중복 제거**: 동일 노드의 중복 서명 방지

3.4 네트워크 통신

3.4.1 메시지 타입

```
pub enum PrimaryMessage {
Header(Header), // 헤더 전파
Vote(Vote), // 헤더에 대한 투표
Certificate(Certificate), // 완성된 인증서
CertificateRequest(CertificateDigest), // 인증서 요청
}
```

3.4.2 통신 패턴

- **Broadcast**: 헤더를 모든 Primary에게 전파
- **Point-to-Point**: 개별 노드와의 직접 통신
- **Request-Response**: 누락된 인증서 요청/응답

3.5 동기화 메커니즘

3.5.1 인증서 동기화

```
pub struct Synchronizer {
network: PrimaryNetwork,
store: CertificateStore,
```

```
gc_depth: Round,
}

impl Synchronizer {
pub  async  fn  sync_certificates(&mut  self,
target_round: Round
) -> Result<(), SyncError> {
// 누락된 인증서 식별
let  missing = self.identify_missing_certificates(target_round)?;
// 병렬로 인증서 요청
let  certificates = self.fetch_certificates(missing).await?;
// 인과관계 순서로 정렬 후 삽입
self.insert_certificates_in_order(certificates).await?;
Ok(())
}
}
```

3.5.2 동기화 전략

- **Lazy Sync**: 필요시에만 동기화 수행
- **Batch Sync**: 여러 인증서를 한 번에 동기화
- **Priority Sync**: 중요한 인증서 우선 동기화

3.6 상태 관리

3.6.1 Primary 상태

```
pub struct PrimaryState {
current_round: Round,
last_committed_round: Round,
pending_headers: HashMap<HeaderDigest, PendingHeader>,
dag: Dag,
committee: Committee,
}
```

3.6.2 상태 전이

- **Round Advancement**: 새로운 라운드로의 진행
- **Epoch Change**: 위원회 변경 시 상태 초기화
- **Recovery**: 장애 복구 시 상태 복원

3.7 성능 최적화

3.7.1 배치 처리

- **Header Batching**: 여러 헤더를 묶어서 처리
- **Signature Batching**: 서명 검증의 배치 처리
- **Network Batching**: 네트워크 메시지 묶음 전송

3.7.2 캐싱 전략

```rust
pub struct PrimaryCache {
header_cache: LruCache<HeaderDigest, Header>,
certificate_cache: LruCache<CertificateDigest, Certificate>,
signature_cache: LruCache<(HeaderDigest, PublicKey), Signature>,
}
```

3.8 장애 처리

3.8.1 네트워크 장애

- **Timeout Handling**: 응답 시간 초과 처리
- **Retry Logic**: 실패한 요청의 재시도
- **Circuit Breaker**: 연속 실패 시 회로 차단

3.8.2 노드 장애

- **Failure Detection**: 장애 노드 감지
- **View Change**: 리더 변경 메커니즘
- **Recovery Protocol**: 복구된 노드의 재참여

3.9 메트릭과 모니터링

3.9.1 성능 메트릭

```rust
pub struct PrimaryMetrics {
headers_created: Counter,
```

```
certificates_created: Counter,

sync_requests: Counter,

round_duration: Histogram,

}
```

3.9.2 모니터링 포인트

- **처리량**: 초당 처리되는 헤더/인증서 수
- **지연시간**: 헤더 생성부터 인증서 완성까지의 시간
- **네트워크 사용량**: 송수신 메시지 크기와 빈도

Primary 노드는 이러한 복잡한 로직들을 통해 높은 성능과 안정성을 제공합니다.